```
$ kubectl get svc -l app=catalog-service

NAME              TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
catalog-service   ClusterIP   10.102.29.119   <none>        80/TCP    42s
```

Since it's of type `ClusterIP`, the Service makes it possible for other Pods within the cluster to communicate with the Catalog Service application, either using its IP address (called the cluster IP) or through its name. That will be useful for the applications you'll build in the next chapters, but what about us? How can we expose the application outside the cluster to test it?

For now, we'll rely on the port-forwarding feature offered by Kubernetes to expose an object (in this case, a Service) to a local machine. You did that already in chapter 2, so the command should look familiar:

```
$ kubectl port-forward service/catalog-service 9001:80
Forwarding from 127.0.0.1:9001 -> 9001
Forwarding from [::1]:9001 -> 9001
```

You can finally call the application from localhost on port 9001, and all requests will be forwarded to the Service object and ultimately to the Catalog Service Pod. Try visiting http://localhost:9001 from your browser to see the welcome message, or http://localhost:9001/books to browse the books available in the catalog.

TIP The process started by the `kubectl port-forward` command will keep running until you explicitly stop it with Ctrl-C. Until then, you'll need to open another Terminal window if you want to run CLI commands.

Figure 7.11 illustrates how the communication works between your computer, Catalog Service, and PostgreSQL.